

Convolutional Neural Networks

Convolution, Weight Sharing, and the Architecture That Taught Machines to See

Neural Networks and Deep Learning Basics · Foundations of Machine Learning

Dense layers do not scale to images — today we derive, piece by piece, the architecture that does

The Cost of Ignoring Image Structure

$$224 \times 224 \times 3 = 150,528 \text{ inputs}$$

A single dense hidden layer

150,528 inputs fully connected to 1,000 hidden units:

$$150,528 \times 1,000 = 150,528,000$$

weights, plus 1,000 biases — for one layer, before any output layer exists

Four concrete failures

Compute: 150M multiply-adds per image, per forward pass

Data hunger: that many free parameters need a comparably enormous labeled dataset to avoid overfitting

No spatial structure: pixel (1,1) and pixel (224,224) are just two entries in a flat vector — "neighbor" has no meaning

No translation invariance: a cat in the top-left and the same cat in the bottom-right activate completely different weights

There has to be a better way.

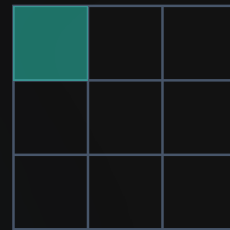
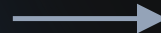
The Convolution Operation

A small filter (kernel) slides across the input, computing a dot product at every position

- Place the kernel at the top-left corner of the input
- Multiply each kernel weight by the pixel value underneath it
- Sum the products into a single output value
- Slide the kernel one step (the **stride**) and repeat across the whole input



5×5 input · 3×3 kernel



3×3 output feature map

$$Y_{i,j} = \sum_{u=0}^2 \sum_{v=0}^2 K_{u,v} X_{i+u, j+v}$$

Convolution, Worked in Numbers

A 4×4 input patch and a Sobel-style vertical edge kernel:

Input X

2	4	1	3
1	5	2	0
3	6	2	1
0	1	4	2

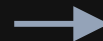
Kernel K

-1	0	1
-2	0	2
-1	0	1

Top-left 3×3 patch, position (0,0)

$$\begin{aligned} &(2)(-1) + (4)(0) + (1)(1) \\ &+ (1)(-2) + (5)(0) + (2)(2) \\ &+ (3)(-1) + (6)(0) + (2)(1) \end{aligned}$$
$$= -2 + 0 + 1 - 2 + 0 + 4 - 3 + 0 + 2 = 0$$

2	4	1	3
1	5	2	0
3	6	2	1
0	1	4	2



0

output (0,0)

Weight Sharing — The Same Filter, Everywhere

Dense layer

224×224×3 image → 1,000 hidden units

150,528,000

independent weights — a fresh weight for every (pixel, unit) pair

Conv layer: 32 filters, 3×3, 3 input channels

$3 \times 3 \times 3 \times 32 + 32 = 896$

896

total learnable parameters — every filter is reused at every spatial location

$150,528,000 / 896 \approx 168,000\times$ fewer parameters

Why fewer parameters is possible

An edge is worth detecting anywhere in the image, not only in the top-left corner — so one small set of weights, reused, does the job of millions of independent ones.

Why it gives translation invariance

Because the identical filter slides over every position, the same feature detector fires wherever the pattern appears — a cat's ear in the corner produces the same response as a cat's ear in the center.

The Feature Hierarchy

Nobody hand-designs these detectors — they emerge from backpropagation

Depth	What it detects
Layer 1	Edges, oriented gradients, color blobs
Layer 2–3	Textures, corners, simple curves
Layer 4–5	Object parts — eyes, wheels, fur patterns
Deep layers	Whole objects, in context

Zeiler & Fergus, "Visualizing and Understanding Convolutional Networks" (2014) — reconstructed what each layer responds to using a "deconvnet"

Emergence, not design

This hierarchy was never specified by engineers. It falls out automatically from training on labeled images with backpropagation and gradient descent.

Reusable low-level features

Edges and curves are useful for recognizing cats, cars, and buildings alike — the reason pretrained CNNs transfer well to new tasks.

Receptive field grows with depth

A single 3×3 kernel only sees a 3×3 patch, but stacking layers lets deeper units respond to much larger regions of the original input.

Max Pooling — A Worked Example

Divide into 2×2 regions; keep only the maximum from each — no weights to learn

Input (4×4)

1	3	2	4
5	6	1	2
3	1	4	3
2	2	1	0

2×2 max pool, stride 2

Top-left: $\{1, 3, 5, 6\} \rightarrow 6$

Top-right: $\{2, 4, 1, 2\} \rightarrow 4$

Bottom-left: $\{2, 2, 1, 3\} \rightarrow 3$

Bottom-right: $\{1, 0, 4, 3\} \rightarrow 4$

Output (2×2)

6	4
3	4

Downsampling

16 values become 4 — a $4 \times$ reduction in compute and memory downstream.

Local translation invariance

A small shift of the strongest activation within a 2×2 window still gives the same max.

Growing abstraction

Each pooled unit now summarizes a larger patch, enlarging the effective receptive field.

The Full CNN Architecture

Raw pixels in, class probabilities out

Input $\rightarrow$ [Conv + ReLU + Pool] $\times$ n $\rightarrow$ Flatten $\rightarrow$ FC $\rightarrow$ Softmax

Input: 32×32×3 image (height × width × RGB channels)

Block 1: Conv(16 filters, 3×3) + ReLU $\rightarrow$ 32×32×16, then 2×2 MaxPool $\rightarrow$ 16×16×16

Block 2: Conv(32 filters, 3×3) + ReLU $\rightarrow$ 16×16×32, then 2×2 MaxPool $\rightarrow$ 8×8×32

Block 3: Conv(64 filters, 3×3) + ReLU $\rightarrow$ 8×8×64, then 2×2 MaxPool $\rightarrow$ 4×4×64

Flatten: 4×4×64 $\rightarrow$ vector of 1,024 values

Dense(128) + ReLU $\rightarrow$ Dense(10) + Softmax — probability over 10 classes

spatial size shrinks, channel depth grows



32×32×3

Recap — Every Problem, Solved by a Named Mechanism

Each CNN design choice traces directly back to a failure of dense networks

Problem: parameter explosion

150,528,000 weights for one dense layer

→ solved by **weight sharing**: 896 parameters for a 32-filter conv layer, reused at every position

Problem: no spatial structure

Flattening destroys pixel neighborhoods

→ solved by **local receptive fields**: each kernel only ever looks at a small spatial patch

Problem: no translation invariance

A shifted cat looks like an unrelated pattern

→ solved by **shift-equivariant convolution + pooling**: the same filter fires wherever the pattern appears

Problem: data hunger

Millions of free parameters need proportionally more labeled data

→ solved by **far fewer parameters**: dramatically less data required to fit the same model reliably

CNNs are not just more accurate on images — they are structurally matched to what images are.

Thank You

Questions & Discussion

Convolutional Neural Networks · Neural Networks and Deep Learning Basics